

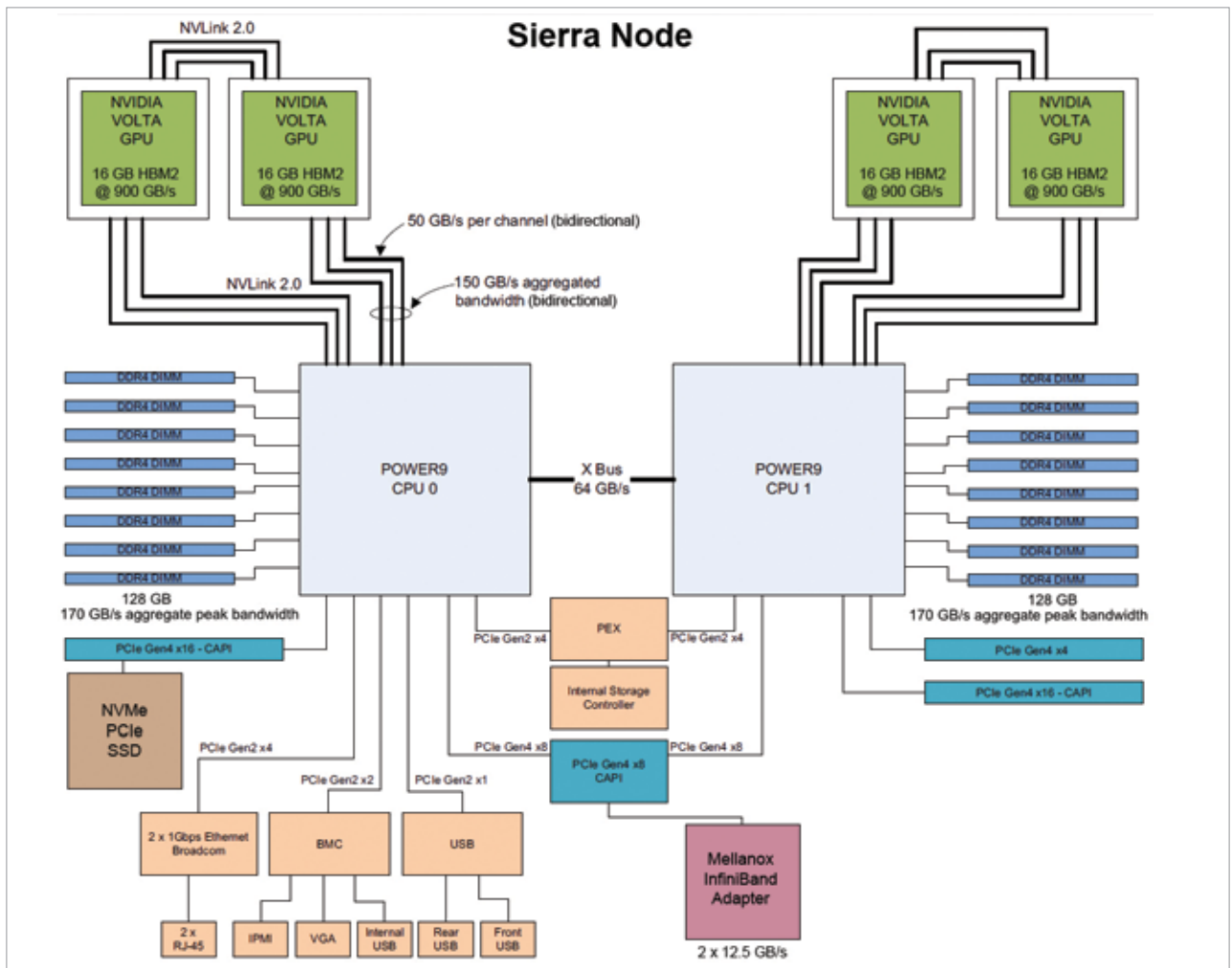


Figure 1: High-performance AI architecture

```

model = mobilenet_v2(weights="DEFAULT").
to(device).eval()
dummy_input = torch.randn(1, 3, 224,
224, device=device)
with torch.no_grad():
    _ = model(dummy_input)
print(f"Inference executed on {device}")

```

GPU acceleration on Windows: CUDA and DirectML

New age AI models require huge computational power, so the use of GPUs for acceleration has become a necessity during both training and inference. Windows supports two main methods: CUDA for NVIDIA GPUs, and DirectML for all-device accepted acceleration.

CUDA-based acceleration: Using CUDA, NVIDIA GPUs can carry out parallel computations, accelerating deep learning tasks. PyTorch and TensorFlow are able to work closely with CUDA on Windows, guaranteeing almost the full GPU performance of a native application.

Here's an example of GPU-accelerated matrix multiplication with PyTorch:

```

import torch
a = torch.randn(1000, 1000,
device="cuda")
b = torch.randn(1000, 1000,
device="cuda")
c = torch.matmul(a, b)
print("Matrix multiplication completed

```

on GPU")

Direct ML: DirectML hides the details of the specific hardware used and thus allows acceleration on NVIDIA, AMD, and Intel GPUs. Usually paired with ONNX Runtime, DirectML is the best choice for inference tasks in business settings that have mixed GPU configurations.

Here's how ONNX Runtime can be paired with DirectML:

```

import onnxruntime as ort
providers = ["DmlExecutionProvider"]
session = ort.InferenceSession("model.
onnx", providers=providers)
print("Using providers:", session.get_
providers())

```